

Kneo Agent Platform

Operation Guide

Version 0.8.0

2026-06-09

Deployment shapes, TLS termination, security hardening, observability, backup and recovery, upgrade, incident response, troubleshooting, and the deployment smoke test — one printable operator guide.

Table of contents

Table of contents	2
Deployment	6
Container	6
Compose	7
TLS and reverse proxy	7
Choosing a persistence backend	7
Readiness and liveness	8
Workers, scaling, and graceful shutdown	8
TLS and reverse proxy	9
Topology	9
Bind address	9
Trusted-proxy headers	10
Reverse-proxy snippets	10
nginx	10
Caddy	10
AWS ALB / generic L7 load balancer	11
Health-check endpoints behind the proxy	11
Verifying TLS is actually in front	11
What kneo-serv does not provide	12
Security hardening	13
Pre-launch checklist	13
1. Enable authentication	13
2. Assign the narrowest role	13
3. Rotate keys without downtime	14
4. Sign spec bundles for production	14
5. Terminate TLS upstream	14
6. Lock down container and host	15
7. Protect the audit trail	15
8. Keep redaction in place	15
Image vulnerability scanning	15
Operator-side verification	16
Accepted findings	16
What kneo-serv deliberately does not provide	17
Observability	18
Three signals, three surfaces	18
Prometheus /metrics	18
Structured request logs	19

Shape	19
Configuration	20
Production tuning	20
Log aggregation wiring	21
Service-side trace events	21
Audit-event export	21
OpenTelemetry spans	22
Platform-side spans	22
Exporter configuration	23
What to watch in production	23
What this page does not cover	24
Backup and recovery	25
What needs to be preserved	25
PostgreSQL — production path	25
Take a backup	26
Restore from a backup (destructive — wipes current state)	26
Off-site rotation	26
Data-only restore into a clean volume	26
SQLite — single-host installs	27
Backup frequency	27
Verifying a restore	28
Rolling back after a failed upgrade	28
Disaster recovery checklist	29
What this page does not cover	29
Upgrade guide	30
Versioning	30
Standard upgrade procedure	30
Persistence migrations	31
Spec migrations	31
SDK compatibility	31
Configuration changes	32
CLI changes	32
Version-specific notes	32
0.1.0 — initial release	32
0.2.0 — first public distribution	32
0.2.1 — /healthz version and Docker /app permission fix	33
0.2.2 — FastAPI info.version fix + post-0.2.0 docs sweep	34
0.3.0	34
0.4.0	35
0.5.0	36

0.6.0	37
0.7.0	39
0.8.0	40
Rolling back	41
Reporting upgrade issues	42
Incident response	43
Triage tree	43
/readyz failure matrix	43
Common production incidents	44
What to capture before escalating	45
When to roll back	46
What this page does not cover	46
Troubleshooting	47
1. Service won't start or won't accept traffic	47
1.1 RuntimeError: KNEO service auth is enabled but no API keys are configured	
1.2 /readyz returns 503	47
1.3 RuntimeError: PlatformManager has not been configured	47
1.4 RuntimeError: Invalid KNEO_SERV_API_KEYS entry	48
2. Persistence and store failures	48
2.1 PostgreSQL DSN configured but service falls back to SQLite	48
2.2 psycopg.OperationalError on startup or first request	48
2.3 SQLite database is locked errors	48
2.4 Schema migration appears to have run but old data is missing	49
2.5 Backup/restore mismatch	49
3. Secrets, credentials, and provider integration	49
3.1 MissingSecretError on agent run	49
3.2 GET /readyz reports missing provider/MCP secrets	49
3.3 kneo spec bundle verify fails	50
4. Authentication and authorization	50
4.1 401 Unauthorized — A valid Kneo service API key is required	50
4.2 403 Forbidden — Missing required scope: <scope>	
4.3 Health endpoints work, all other routes 401	50
5. Run lifecycle problems	50
5.1 Async runs sit in queued and never progress	50
5.2 Cancelled run still finishes as succeeded	51
5.3 Run hangs at a workflow step	51
5.4 409 Conflict — idempotency_key_conflict	51
5.5 400 Bad Request — invalid_idempotency_key	51
6. Spec validation and compilation	52
6.1 SpecCompilationError with diagnostics	52

6.2 ValueError: Tool '<name>' has no implementation	
6.3 Inline spec rejected with size error	52
7. Observability	52
7.1 Structured logs missing request_id	52
7.2 OpenTelemetry not exporting	52
7.3 Trace events missing for a run	53
8. Human-in-the-loop	53
8.1 LockAcquisitionError on resume	53
8.2 Continuation expired or missing	53
9. Release and supply chain	53
What to capture before opening a bug	54
Deployment smoke test	55
Compose stack	55
PostgreSQL coverage	56
Staging and remote smoke	56
See also	57

Deployment

Source: <docs/user/deployment.md>

Reference for the supported deployment shapes. For a guided zero-to-running walkthrough on Docker Compose with PostgreSQL, see tutorial_postgres_deployment.md. For every environment variable referenced below, see <environment.md>.

The service supports three shapes:

- **Container** — a single `kneo-serv` image with a database you supply.
- **Compose** — the bundled stack that starts the API plus PostgreSQL.
- **Embedded** — `kneo_serv.service.app:create_app()` mounted in your own ASGI server (covered in [tutorial_custom_tool.md § 7](tutorial_custom_tool.md)).

Container

Pull the published image from GitHub Container Registry:

```
docker pull ghcr.io/kneo-agent/kneo-serv:latest
```

Tag conventions: `<version>` (e.g. `0.7.0`), `<major>.<minor>` (`0.7`), and `latest`. `amd64-only`; `arm64` is deferred to [1.0](#). From 0.3.0 onward the image is keyless-signed via cosign and ships with a CycloneDX SBOM attestation; from 0.4.0 onward the release pipeline also runs a blocking Trivy CVE scan against the pushed digest under the CVSS≥7 policy ([security_hardening.md § Image vulnerability scanning](security_hardening.md)) — verification commands in [supply_chain_review.md § Verification commands](supply_chain_review.md). The image installs the `kneo-serv[deploy]` extra (psycopg + SDK telemetry).

Run it against PostgreSQL:

```
docker run --rm -p 8000:8000 \
  -e KNEO_SERV_DATABASE_URL=postgresql://kneo_serv:change-
me@host.docker.internal:5432/kneo_serv \
  -e KNEO_SERV_AUTH_ENABLED=true \
  -e KNEO_SERV_API_KEYS='operator:replace-token:operator' \
  ghcr.io/kneo-agent/kneo-serv:latest
```

For local builds from a source checkout (contributor / pre-publish):

```
docker build -t kneo-serv:local .
```

Compose

The bundled Compose stack starts the API plus PostgreSQL:

```
cp deploy/production.env.example deploy/production.env
docker compose --env-file deploy/production.env up --build
```

Replace every placeholder token and database password before binding the service to a network. The stack defaults to port `8000`; set `KNEO_SERV_PORT` to change the host-side port.

For a staging rehearsal, use the staging env example:

```
cp deploy/staging.env.example deploy/staging.env
KNEO_SERV_ENV_FILE=./deploy/staging.env \
  docker compose --env-file deploy/staging.env up --build
```

`deploy/staging.env` is gitignored. Keep SDK telemetry argument/result capture (`KNEO_SERV_OTEL_RECORD_ARGUMENTS` , `KNEO_SERV_OTEL_RECORD_RESULTS`) disabled in staging unless the deployment's data classification has explicitly approved payload capture.

TLS and reverse proxy

The service speaks bare HTTP and does not terminate TLS itself. For any deployment exposed beyond `127.0.0.1`, place a reverse proxy (nginx, Caddy, AWS ALB, or similar) in front and terminate TLS there. See [tls_and_proxy.md](#) for topology, bind-address guidance, and trusted-proxy header handling.

Choosing a persistence backend

Backend	When to use
SQLite	Local dev or single-process service. Default when <code>KNEO_SERV_DATABASE_URL</code> is unset.
PostgreSQL	Any multi-process or production deployment. Set <code>KNEO_SERV_DATABASE_URL</code> . Requires <code>kneo-serv[postgres]</code> or <code>kneo-serv[deploy]</code> .

When `KNEO_SERV_DATABASE_URL` is set, the service uses PostgreSQL for run state, checkpoints, idempotency records, queue leases, locks, audit events, and workflow continuations. Without it, the service falls back to SQLite for state and file-backed continuations.

Multi-process SQLite is not a supported topology; see [troubleshooting.md § 2.3](#).

For the throughput and latency trade-offs between the two backends — including why SQLite write throughput does not scale with concurrency — and a bench harness to size your own deployment, see [performance.md](#).

Readiness and liveness

Wire these endpoints into your supervisor or load balancer:

```
GET /livez      # process liveness
GET /readyz    # readiness: all dependencies healthy
GET /healthz   # lightweight overall health
```

`/livez` and `/readyz` are intentionally unauthenticated for probe integration. `/readyz` returns `503` with a structured `not_ready` payload when any dependency check fails — see [troubleshooting.md § 1.2](#) for the failure shape.

The Prometheus scrape endpoint `GET /metrics` (since 0.5.0) is also unauthenticated — restrict it to your monitoring network or disable it with `KNEO_SERV_METRICS_ENABLED=false`. See [observability.md § Prometheus /metrics](#).

Workers, scaling, and graceful shutdown

A single process runs a pool of `KNEO_SERV_WORKER_CONCURRENCY` worker threads (default `1`) draining the run queue. Raise it for provider-bound workloads; for write-concurrent scale on PostgreSQL, run **multiple service processes** — each leases queued runs safely via `FOR UPDATE SKIP LOCKED`. Sizing guidance and the SQLite single-writer caveat are in [performance.md](#).

On `SIGTERM` (e.g. a rolling deploy) the service drains the worker pool: workers stop claiming new work and finish the run they are currently executing, and the service waits up to `KNEO_SERV_SHUTDOWN_TIMEOUT_SECONDS` (default `30`) for them to exit. A run *still executing* when that timeout elapses is interrupted by process exit — but it stays claimed and is automatically re-leased and retried by another worker once its `KNEO_SERV_WORKER_LEASE_SECONDS` lease expires, so it is not lost, only restarted. To drain in-flight runs without that restart, set `KNEO_SERV_SHUTDOWN_TIMEOUT_SECONDS` (and your orchestrator's termination grace period) at least as long as your longest expected run step. Set `KNEO_SERV_MAX_QUEUE_DEPTH` to shed load with `503` under overload, and `KNEO_SERV_QUEUE_MAX_ATTEMPTS` (default `5`) to dead-letter poison runs rather than retry them forever.

TLS and reverse proxy

Source: docs/user/tls_and_proxy.md

The Kneo Agent Platform service speaks plain HTTP. It does not terminate TLS, parse `X-Forwarded-*` headers itself, or rate-limit by IP. Any deployment that faces a network beyond `127.0.0.1` must run behind a reverse proxy that terminates TLS and shields the service.

For deployment shapes (Container, Compose, Embedded) and the choice of persistence backend, see <deployment.md>. For the hardening checklist that includes TLS, see security_hardening.md.

Topology

```

client —HTTPS→ reverse proxy —HTTP→ kneo-serv
                (TLS termination,
                 request size limits,
                 rate limiting,
                 client-IP injection)

```

The proxy is responsible for:

- TLS termination and certificate management
- Request body size limits at the edge (defense in depth above `KNEO_SERV_MAX_BODY_BYTES`)
- IP-based rate limiting if you need it (the service has no built-in per-IP limiter)
- Forwarding the client IP for the service's structured logs

Run the proxy and `kneo-serv` on the same host (or in the same private network) so the unencrypted hop is not exposed.

Bind address

Topology	<code>--host</code> value	Rationale
Proxy + service on the same host	<code>127.0.0.1</code>	Service is unreachable except through the proxy.
Proxy + service in a shared private network	<code>0.0.0.0</code>	The network boundary is the proxy; firewall the service port.
Compose (<code>compose.yaml</code>)	<code>0.0.0.0</code> inside the container; only the proxy's port is published on the host.	The Compose stack's internal network already isolates the API service.

The Dockerfile defaults to `--host 0.0.0.0 --port 8000`. Override with `KNEO_SERV_PORT` for the published host-side port; the container port is fixed at `8000`.

Trusted-proxy headers

The service logs the immediate TCP peer as `client_ip` in its structured request logs ([observability.md](#)). When a proxy fronts the service, the immediate peer is the proxy, not the original client. To capture the real client IP in logs and traces, configure the proxy to write `X-Forwarded-For` upstream and ingest it at your log aggregator — the service itself does not rewrite `client_ip` from `X-Forwarded-For` (no implicit trust).

The service does honor `X-Request-ID` and echoes it back on the response. Proxies that already inject a request ID should pass it through; the service generates a UUID per request otherwise.

Reverse-proxy snippets

These are minimal examples. Production configurations should add timeouts, buffer sizing, and rate-limit zones; consult your proxy's docs.

nginx

```
server {
    listen 443 ssl http2;
    server_name kneo.example.com;

    ssl_certificate      /etc/letsencrypt/live/kneo.example.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/kneo.example.com/privkey.pem;

    client_max_body_size 2m;    # match or exceed KNEO_SERV_MAX_BODY_BYTES

    location / {
        proxy_pass          http://127.0.0.1:8000;
        proxy_http_version 1.1;
        proxy_set_header    Host                $host;
        proxy_set_header    X-Forwarded-For     $proxy_add_x_forwarded_for;
        proxy_set_header    X-Forwarded-Proto   $scheme;
        proxy_set_header    X-Request-ID        $request_id;
        proxy_read_timeout   130s;               # exceed KNEO_SERV_CLIENT_TIMEOUT
    }
}
```

Caddy

```
kneo.example.com {
  reverse_proxy 127.0.0.1:8000 {
    header_up X-Forwarded-For {remote_host}
    header_up X-Request-ID    {http.request.uuid}
  }
  request_body {
    max_size 2MB
  }
}
```

AWS ALB / generic L7 load balancer

- Listener: HTTPS 443 with an ACM certificate; redirect 80 → 443.
- Target group: HTTP, port 8000, healthcheck GET /readyz (interval 30s, unhealthy threshold 3, success codes 200). /readyz is unauthenticated by design.
- Idle timeout: ≥ KNEO_SERV_CLIENT_TIMEOUT (default 120s); set 130s for a safety margin.
- Body size: ALBs cap at 1 MiB by default — if you accept larger inline specs (KNEO_SERV_MAX_INLINE_SPEC_BYTES is 256 KiB by default), use a CloudFront or nginx tier in front and bypass the ALB cap accordingly.

Health-check endpoints behind the proxy

Expose /livez and /readyz directly to the proxy or load balancer. Both are unauthenticated to keep probe integration simple. Do **not** expose /readyz to the public internet — its 503 not_ready payload includes internal check names and registry contents that should stay inside the operational perimeter.

For most setups: bind the proxy's probe routes to internal listeners only, or restrict the source IP range to your load-balancer subnet.

Verifying TLS is actually in front

```
# TLS terminates at the proxy, service is unreachable directly.
curl -sf https://kneo.example.com/readyz | jq '.metadata.ready' # → true
curl -sf http://kneo.example.com/readyz                       # → connection refused / 301
curl -sf http://127.0.0.1:8000/readyz                         # → only succeeds from the
proxy host
```

If the third command succeeds from outside the proxy host, the service port is reachable from the public network and the bind address or firewall is misconfigured.

What `kneo-serv` does not provide

- **No built-in TLS.** Terminate at the proxy.
- **No `X-Forwarded-For` rewriting.** Capture client IPs at the proxy or in your log aggregator.
- **No per-IP rate limiting.** Use the proxy's rate-limit zone.
- **No mTLS to upstream providers.** Provider connections go out from the service host; lock down egress at the network layer.

See [security_hardenning.md](#) for the full pre-launch checklist.

Security hardening

Source: docs/user/security_hardening.md

Pre-launch checklist for taking a `kneo-serv` deployment to production. Each item references the authoritative configuration doc; this page is the single sheet to walk before going live.

For the auth model itself (roles, scopes, route mapping), see service_api.md § Authentication.
For audit-event details, see service_api.md § Audit events.

Pre-launch checklist

1. Enable authentication

- ☐ `KNEO_SERV_AUTH_ENABLED=true` is set (or `KNEO_SERV_API_KEYS` / `KNEO_SERV_ADMIN_API_KEY` are set, which enables auth implicitly).
- ☐ No API keys are committed to the repo, the Compose `.env` file, or example configs.
- ☐ Each consumer has its own key, named so audit events identify the caller (`KNEO_SERV_API_KEYS='ci:...:service;analyst:...:viewer'`).
- ☐ The admin key (`KNEO_SERV_ADMIN_API_KEY`) is issued separately and used only for break-glass operations.

2. Assign the narrowest role

Pick the narrowest built-in role that covers each caller's needs. The canonical role-to-scope mapping lives in service_api.md § Authentication; below is the operational guidance for choosing between them.

Role	Use for
<code>admin</code>	Break-glass operator key only
<code>operator</code>	Day-to-day operator console / CI deploy
<code>service</code>	Server-to-server callers that drive runs
<code>reviewer</code>	Human-in-the-loop approvers
<code>viewer</code>	Dashboards, read-only analytics

Custom scopes are allowed in the third field of `KNEO_SERV_API_KEYS` when no built-in role fits.

- ☐ No consumer is using `admin` for routine traffic.

- [] Read-only consumers are on `viewer`, not `operator`.

3. Rotate keys without downtime

`kneo-serv` has no in-place key rotation API in the 0.x (pre-1.0) line. Rotation is a config swap:

1. Add the new key to `KNEO_SERV_API_KEYS` alongside the old key (semicolon-separated entries; same `name:` is fine).
 2. Restart the service. Both keys are now valid.
 3. Roll callers over to the new key.
 4. Remove the old entry from `KNEO_SERV_API_KEYS`.
 5. Restart again.
- [] Rotation procedure is rehearsed in staging before production keys are issued.
 - [] Old keys are revoked, not left "in case."

4. Sign spec bundles for production

For environments that block ad-hoc spec edits:

- [] `KNEO_SERV_SPEC_SIGNING_KEY` is set in CI and on the service hosts (different value than any API key).
- [] Production deploys use signed bundles only: `kneo spec bundle sign ... --approved-by <name> --env prod` and `kneo spec bundle verify <bundle>` in the deploy pipeline.
- [] Project config declares `environments.prod.policy_enforcement` so CLI and API spec flows enforce policy after overlays ([project_config.md](#)).

spec_path and overlays are filesystem-trusted inputs. A request that supplies `spec_path` (or overlay paths) makes the service read those files from its own filesystem. This is intentional and matches how a deployed service loads its specs, but it means the *caller of those paths must be trusted* — anyone who can set `spec_path / overlays` can cause the service to read any file path it has access to. Restrict who can submit path-bearing spec requests to operators, and prefer inline `spec` (with `overrides`) for less-trusted callers. Spec *content* validation (`/v1/specs/validate`) stays pure and performs no filesystem I/O.

5. Terminate TLS upstream

- [] A reverse proxy in front of the service terminates TLS; the service bind address is `127.0.0.1` or restricted to a private network. See [tls_and_proxy.md](#).
- [] `/readyz` is exposed only to the load balancer or probe subnet (see [tls_and_proxy.md § Health-check endpoints behind the proxy](#)).
- [] The proxy enforces a request body size limit $\geq$ `KNEO_SERV_MAX_BODY_BYTES`.

6. Lock down container and host

The bundled Dockerfile already enforces a non-root user (`kneo`); you do not need to override it. ([Dockerfile](#))

- [] Container runs as the non-root `kneo` user (default).
- [] Image is pulled from a trusted registry; tags are pinned by digest in production manifests.
- [] CI scans the image for known CVEs; releases blocked on HIGH/CRITICAL findings ([§ Image vulnerability scanning](#)).
- [] Host or Kubernetes drops Linux capabilities the service doesn't need (no `NET_ADMIN` , no `SYS_ADMIN`).
- [] Egress is restricted at the network layer to the provider, MCP, and observability endpoints the deployment actually uses.

The container's filesystem is **not** read-only — the service writes checkpoints, queue state, and optionally SQLite files. If you need read-only-root, mount writable volumes for `.kneo/` (SQLite + continuations) and the artifact paths declared in your spec.

7. Protect the audit trail

- [] Audit events are persisted in the same backend as run state (PostgreSQL in production); the DB is backed up per [backup_and_recovery.md](#) .
- [] `audit:read` is scoped to a small set of principals (compliance, on-call, incident-response).
- [] Audit-event retention is set deliberately. If `KNEO_SERV_RETENTION_RUNS_DAYS` is set, runs and their audit events age out together — confirm that aligns with your compliance window before enabling it ([environment.md § Retention](#)).

8. Keep redaction in place

`kneo-serv` redacts secrets, tokens, authorization headers, emails, and SSNs from responses, traces, checkpoints, and CLI JSON output by default ([service_api.md § Redaction](#)). The two escape hatches both default to off:

- [] `KNEO_SERV_OTEL_RECORD_ARGUMENTS=false` (unless tool arguments are classified safe to emit to your trace backend).
- [] `KNEO_SERV_OTEL_RECORD_RESULTS=false` (same rationale).
- [] Custom tools and middleware do not log user inputs or provider responses without redaction.

Image vulnerability scanning

The release pipeline scans every published GHCR image for known CVEs using [Trivy](#). The scan runs against the **pushed image digest** (the same bytes cosign signed and SBOM attestation describes), so the four supply-chain artifacts — image, cosign signature, SBOM attestation, and scan report — all agree on what they describe.

Locked policy (0.4.0; recorded in [archive_TODO-0.4.0.md](#)):

- **Severity threshold:** CVSS $\geq$ 7 (HIGH and CRITICAL findings).
- **Release-tag scans (`v<version>` / `v<version>rcN`):** blocking. The Trivy step in [release.yml](#) runs with `--exit-code 1`; HIGH/CRITICAL findings fail the step, preventing the Publish build artifact and Publish GitHub release steps from running. The publish is the irreversible step, so the gate fires there.
- **PR-time scans:** report-only via [.github/workflows/image-scan.yml](#). The same scanner version + severity threshold runs against a locally-built image but with `--exit-code 0`, so findings surface in the PR's check summary without blocking merges. Dev velocity isn't gated on un-fixable transient base-image CVEs; the release gate catches anything that matters before publish.
- **Scan report retention:** the JSON report is attached as a GitHub Actions artifact (`trivy-report-<version>`) on every release-tag build, retained for 90 days. The deployer can download it for audit.

Operator-side verification

Re-run the scan locally against any published tag:

```
trivy image \
  --severity HIGH,CRITICAL \
  --ignore-unfixed=false \
  ghcr.io/kneo-agent/kneo-serv:<tag>
```

Cross-check against the release-time scan output by downloading the `trivy-report-<version>` artifact from the GitHub Release.

Accepted findings

If an upstream CVE has no fix available, or the deployer's risk tolerance accepts a specific finding (e.g. low exploitability in your network posture), record the acceptance in [supply_chain_review.md](#) & [Current workspace result](#) using the same shape as the existing `pip-audit` remediation blocks. For *dependency* (`pip-audit`) findings the release pipeline has no inline-ignore mechanism — those acceptances are deployer policy. For *image* CVEs, the blocking Trivy gate (since 0.4.0) reads acceptances from `.trivyignore` in the repo root, the active image-CVE-acceptance policy.

What `kneo-serv` deliberately does not provide

Operators sometimes go looking for these; document the gap rather than inventing it:

- **No built-in TLS.** Terminate at a reverse proxy ([tls_and_proxy.md](#)).
- **No per-IP rate limiting.** Use the reverse proxy's rate-limit zone.
- **No mTLS to upstream providers.** Provider HTTPS calls leave the service host; control with egress firewall rules.
- **No live key rotation API.** Keys are configured via env vars; rotate with a config swap and restart (§ 3).
- **No external secret-manager integration.** Secrets are injected via environment variables. Use your platform's secret store (Kubernetes Secrets, AWS Secrets Manager, Vault) to populate the env at startup.
- **No SCIM or directory integration.** API keys are flat-file in `KNEO_SERV_API_KEYS` ; map them to identities in your audit log aggregator.

These are tracked in the roadmap, not bugs. See [docs/plan/roadmap.md](#) .

Observability

Source: <docs/user/observability.md>

Operator guide for wiring `kneo-serv`'s structured logs, request tracing, and OpenTelemetry exports into a production observability stack.

This page is the setup view. For symptoms and recovery when observability itself misbehaves, see <troubleshooting.md> § 7. For the full env-var list, see <environment.md> § Observability.

Three signals, three surfaces

Signal	What it is	Where it comes from
Structured request logs	One JSON record per HTTP request, redacted	<code>RequestLoggingMiddleware</code> (always on by default)
Service-side trace events	Per-run trace and checkpoint records, queryable via the API	<code>TracingMiddleware</code> , exposed at <code>/v1/runs/{run_id}/trace</code>
OpenTelemetry spans	Distributed-tracing spans across SDK-driven agent / tool calls and platform-side operations (queue dispatch, worker lease, continuation lock)	<code>kneo_agent.observability.OpenTelemetryMiddleware</code> + <code>kneo_serv.observability.platform_tracer</code> , opt-in via <code>KNEO_SERV_OTEL_ENABLED</code>
Prometheus metrics	Run-queue gauges + per-process run & token counters	<code>GET /metrics</code> (since 0.5.0), opt-out via <code>KNEO_SERV_METRICS_ENABLED</code>

Prometheus `/metrics`

Since 0.5.0 the service exposes a Prometheus scrape endpoint at `GET /metrics` (root path only — not under `/v1`). It is **unauthenticated**, like `/healthz`: it carries operational counts, not run content or secrets. Mount it only on a network your monitoring stack can reach (bind the service behind a reverse proxy that does not expose `/metrics` publicly), or disable it with `KNEO_SERV_METRICS_ENABLED=false`.

Metric	Type	Meaning
<code>kneo_runs_started_total</code>	counter	Runs that began execution.

Metric	Type	Meaning
<code>kneo_runs_completed_total</code>	counter	Runs that completed successfully.
<code>kneo_runs_failed_total</code>	counter	Runs that failed / timed out / hit max-iterations.
<code>kneo_runs_dead_lettered_total</code>	counter	Runs dead-lettered after exceeding <code>KNEO_SERV_QUEUE_MAX_ATTEMPTS</code> .
<code>kneo_tokens_input_total</code>	counter	Input/prompt tokens consumed across runs (from each run's <code>metadata["usage"]</code> , when the runtime reports it).
<code>kneo_tokens_output_total</code>	counter	Output/completion tokens produced across runs.
<code>kneo_tokens_total</code>	counter	Total tokens (input + output) across runs.
<code>kneo_runs_queued</code>	gauge	Runs queued and awaiting a worker (your backlog / backpressure signal).
<code>kneo_runs_running</code>	gauge	Runs currently leased by a worker.
<code>kneo_worker_count</code>	gauge	Live worker threads in this process.

Counters are **per-process** and reset on restart — use `rate()` in Prometheus. In a multi-process deployment each instance exposes its own counters; aggregate across instances with `sum()`. Latency percentiles are not exported here; read them from the OTEL spans or your reverse-proxy access logs.

Structured request logs

Shape

Each request emits a single JSON record on the `kneo_serv.service` logger:

```
{
  "client_ip": "10.0.0.7",
  "duration_ms": 18.214,
  "event": "http_request",
  "method": "POST",
  "path": "/v1/runs",
  "request_id": "f3b3...",
  "run_id": "run_...",
  "status_code": 201,
  "user_agent": "kneo-serv-client/0.2.2"
}
```

Fields the middleware always emits: `event`, `request_id`, `method`, `path`, `status_code`, `duration_ms`. Optional fields when available: `client_ip`, `user_agent`. Route-derived fields when the path includes them: `run_id`, `continuation_id`. When the request raises: `error` (exception class name) and `message` (exception message). Redaction is applied to every payload before it reaches the log line. ([kneo_serv/observability/structured_logging.py](#))

Configuration

Variable	Default	Purpose
<code>KNEO_SERV_REQUEST_LOGS</code>	<code>true</code>	Enable the JSON request log middleware.
<code>KNEO_SERV_LOG_LEVEL</code>	<code>INFO</code>	Stack-wide level: <code>kneo_serv.service</code> + <code>kneo_serv.platform</code> + <code>kneo_agent</code> SDK.

`request_id` is generated server-side as a UUID unless the client sends `X-Request-ID`; either way the service echoes it back on the response header.

Production tuning

- Keep `KNEO_SERV_LOG_LEVEL=INFO` in production. `DEBUG` doubles log volume and can leak diagnostic payloads from middleware that wraps the request logger. Note it is **stack-wide**: it sets the request logger, the platform worker / lease / drain logger (`kneo_serv.platform`), and the `kneo_agent` SDK logger together, so `DEBUG` turns all three up at once — useful for diagnosing a stuck run end-to-end, noisy as a default.
- Configure your container runtime's log driver (Docker `json-file` with rotation, Kubernetes `kubectrl logs rotation`, `journald`) — the service writes to stdout and relies on the runtime to rotate.

Log aggregation wiring

- **ELK / OpenSearch.** Ship stdout via Filebeat or Vector. The records are already JSON; map `request_id` and `run_id` as indexed fields. Pin `service.name=kneo-serv` from the shipper for cross-deployment search.
- **Loki.** A Promtail pipeline with a `json` stage will lift `request_id`, `run_id`, `status_code`, and `duration_ms` to labels. Keep label cardinality bounded — don't promote `request_id` to a Loki label, query it as content.
- **Cloud-managed (CloudWatch Logs, GCP Logging).** Forward stdout; the managed pipeline parses JSON automatically.

The reverse proxy in front of `kneo-serv` ([tls_and_proxy.md](#)) has the true client IP. The service logs the immediate TCP peer; correlate to the proxy's access logs by `request_id` (forward `X-Request-ID` upstream).

Service-side trace events

Service-side trace events are persisted as part of run state and returned at `GET /v1/runs/{run_id}/trace`. They cover workflow step transitions, tool calls, checkpoints, and audit boundaries. These events are emitted by `TracingMiddleware` independent of any OTel exporter, so they are always available even without OpenTelemetry.

See [service_api.md § Audit events](#) and [service_api.md § Replay and checkpoint diff](#) for the contract.

Audit-event export

Audit events are always queryable via `GET /v1/audit-events` and stored in the run-state store. For compliance retention you can additionally **stream them out**: set `KNEO_SERV_AUDIT_EXPORT_ENABLED=true` and each persisted event is emitted as a JSON line on the dedicated `kneo_serv.audit` logger, from the same `record_audit_event` chokepoint (so the payload is already redacted).

It is **off by default** — enabling it is a zero-behavior-change opt-in. The transport is plain stdlib logging, so wire it to your sink the usual way:

```
import logging
from logging.handlers import SysLogHandler

audit = logging.getLogger("kneo_serv.audit")
audit.addHandler(SysLogHandler(address=("siem.internal", 514)))
audit.setLevel(logging.INFO)
```

With no dedicated handler the events propagate to the root logger and land in your normal JSON logs alongside the request logs. This is the v1 sink; a direct SIEM/OTLP-logs exporter may follow.

OpenTelemetry spans

When the deployment includes the SDK telemetry support (the `[telemetry]` or `[deploy]` extras), set `KNEO_SERV_OTEL_ENABLED=true` to attach `kneo_agent.observability.OpenTelemetryMiddleware`. Argument and result capture (`KNEO_SERV_OTEL_RECORD_ARGUMENTS`, `KNEO_SERV_OTEL_RECORD_RESULTS`) are off by default because tool inputs and outputs frequently contain user payloads — enable them only after the deployment's data classification has approved payload capture.

See [environment.md § Observability](#) for the full env-var reference.

Platform-side spans

The SDK's `OpenTelemetryMiddleware` covers the *agent boundary* — runs, tool calls, model calls. The platform also instruments operations that happen *outside* the agent's execution:

Span name	Where	Attributes
<code>kneo.queue.dispatch</code>	<code>PlatformManager.dispatch_run</code> — when a run is enqueued for an async worker	<code>kneo.run.id</code>
<code>kneo.worker.lease</code>	Async worker loop — one span per lease attempt against the queue	<code>kneo.worker.id</code> , <code>kneo.worker.lease_seconds</code> , <code>kneo.worker.claimed</code> (bool), <code>kneo.run.id</code> (if claimed)
<code>kneo.continuation.lock</code>	<code>PlatformManager.resume_human_task</code> — when the per-continuation lock is acquired before resume	<code>kneo.continuation.id</code> , <code>kneo.lock.name</code> , <code>kneo.lock.ttl_seconds</code> , <code>kneo.lock.acquired</code> (bool)

These spans share the same `KNEO_SERV_OTEL_ENABLED` flag — they're a clean no-op when telemetry is off (no overhead beyond a single env-var check). Span names use the `kneo.<area>.<operation>` convention so they sort cleanly alongside SDK-owned spans in tracing UIs.

GenAI semantic conventions (`gen_ai.*`). The OpenTelemetry GenAI attributes (`gen_ai.system`, `gen_ai.operation.name`, token counts, ...) are emitted by the SDK's `OpenTelemetryMiddleware` on the **agent / model-call** spans — that is where the model provider is known. The platform-side spans

above are provider-agnostic infrastructure operations (queue / lease / lock) and deliberately do **not** carry `gen_ai.*`; querying GenAI telemetry uses the SDK-owned model spans.

Lease spans with `kneo.worker.claimed=false` indicate an empty queue — useful for measuring how often workers idle. Continuation lock spans with `kneo.lock.acquired=false` correlate with the `LockAcquisitionError` shown in [troubleshooting.md § 8.1](#).

Exporter configuration

The service uses the OpenTelemetry global tracer provider; exporters are configured with standard `OTEL_*` environment variables that the OTel SDK reads. Example for OTLP/HTTP to any compatible backend (Honeycomb, Grafana Tempo, Tempo Cloud, Datadog, an OTel Collector):

```
export KNEO_SERV_OTEL_ENABLED=true

export OTEL_EXPORTER_OTLP_PROTOCOL=http/protobuf
export OTEL_EXPORTER_OTLP_ENDPOINT=https://api.honeycomb.io
export OTEL_EXPORTER_OTLP_HEADERS="x-honeycomb-team=$HONEYCOMB_API_KEY"
export OTEL_SERVICE_NAME=kneo-serv
export OTEL_RESOURCE_ATTRIBUTES="deployment.environment=prod"
```

For a self-hosted OTel Collector running as a sidecar:

```
export OTEL_EXPORTER_OTLP_ENDPOINT=http://localhost:4317
export OTEL_EXPORTER_OTLP_PROTOCOL=grpc
```

If OTel does not appear to be exporting, see [troubleshooting.md § 7.2](#).

What to watch in production

A minimal alerting baseline covers the failure modes that page on-call:

Signal	What it means	Where to read it
<code>/readyz</code> returns <code>503</code> for more than 1 probe interval	A dependency check is failing	Reverse proxy / load balancer health checks
Sustained <code>5xx</code> rate above baseline	Service-side errors	Proxy access logs; <code>status_code</code> from JSON logs
<code>duration_ms</code> p95 climbing over baseline	Latency regression — provider, queue, or DB pressure	JSON log records

Signal	What it means	Where to read it
Queue depth (<code>status=queued</code>) growing unbounded	Workers stuck or backpressured	<code>/readyz queue check; curl "\$BASE/v1/runs?status=queued&limit=20"</code>
Spike in <code>event=http_request</code> records with <code>error</code>	Application-level exceptions	JSON log records

Wire your alerting against these signals from the proxy and the aggregated logs; the service does not push its own alerts.

What this page does not cover

- **Per-IP rate limiting and traffic shaping.** The reverse proxy's job ([tls_and_proxy.md](#)).
- **Request-level latency histograms on `/metrics`.** The `/metrics` endpoint (since 0.5.0) exports run-queue gauges and run counters, not per-request latency percentiles. Derive request latency from OTel spans or the reverse-proxy access logs.
- **Tracing internals.** For the design of the in-process tracer and checkpoint events, see [docs/dev/design.md](#) and [docs/dev/implementation_map.md](#) .

Backup and recovery

Source: docs/user/backup_and_recovery.md

Production procedure for backing up `kneo-serv` state, verifying restores, and rolling back a deployment. This page consolidates the operator surface; the underlying Python API and SQL commands stay in their respective references.

For the upgrade context that ends in "and keep a backup", see <upgrade.md>. For the Python backup API used by the SQLite maintenance helpers, see [service_api.md \\$ Backup and restore](service_api.md#Backup-and-restore).

What needs to be preserved

State	Where it lives	Backup mechanism
Run state, queue, checkpoints, audit events, idempotency, locks, policies	PostgreSQL (<code>KNEO_SERV_DATABASE_URL</code> set) or SQLite at <code>.kneo/kneo_runs.sqlite</code> (default)	<code>pg_dump</code> / SQLite online-backup
Workflow continuations	PostgreSQL when set, otherwise files under <code>.kneo/continuations/</code>	DB dump or filesystem backup
Spec bundles	Source repo + your CI artifacts (signed bundles)	Repo + artifact store
Artifacts (workflow outputs)	Filesystem paths declared by your specs	Filesystem backup
Logs	stdout via container log driver → log aggregator	Aggregator retention

The DB is the load-bearing piece. Everything else can be reconstructed from the DB and your spec repo, except for filesystem-stored continuations and artifacts when PostgreSQL is not configured.

PostgreSQL — production path

The Compose stack and any production deployment should set `KNEO_SERV_DATABASE_URL`. In that mode all state above (except artifacts) lives in PostgreSQL.

Take a backup

```
docker compose --env-file deploy/production.env exec db \
  pg_dump -U "$POSTGRES_USER" "$POSTGRES_DB" \
  | gzip > "kneo_serv-$(date +%Y%m%d-%H%M).sql.gz"
```

For a host-level Postgres install, run `pg_dump` directly as the `postgres` user; the data shape is the same.

Restore from a backup (*destructive — wipes current state*)

```
gunzip -c kneo_serv-YYYYmmDD-HHMM.sql.gz \
  | docker compose --env-file deploy/production.env exec -T db \
    psql -U "$POSTGRES_USER" "$POSTGRES_DB"
```

Restore replaces every row in the database. Stop the API container first so no in-flight write races the restore:

```
docker compose --env-file deploy/production.env stop api
gunzip -c kneo_serv-YYYYmmDD-HHMM.sql.gz \
  | docker compose --env-file deploy/production.env exec -T db \
    psql -U "$POSTGRES_USER" "$POSTGRES_DB"
docker compose --env-file deploy/production.env start api
```

Off-site rotation

Local backups protect against operator error, not host loss. After each dump, copy the gzip off the host:

- S3, Azure Blob, or GCS bucket with versioning + lifecycle to archive older dumps.
- Encrypt at rest (server-side encryption is sufficient if your control plane is locked down; client-side encryption for stricter regimes).
- Apply a separate IAM identity for upload-only versus read.

Data-only restore into a clean volume

For test-restore drills and disaster recovery into a fresh PostgreSQL volume, the service handles schema migrations on startup. Capture a data-only dump and exclude the `schema_migrations` rows so the new volume's migration state isn't overwritten:

```
docker compose --env-file deploy/production.env exec -T db \
  pg_dump -U "$POSTGRES_USER" -d "$POSTGRES_DB" --data-only --inserts \
  -f /tmp/kneo_serv_data.sql
docker cp <db-container-id>:/tmp/kneo_serv_data.sql /tmp/kneo_serv_data.sql
```

```
grep -v "INSERT INTO public.schema_migrations" /tmp/kneo_serv_data.sql \
> /tmp/kneo_serv_data_restore.sql
```

Restore into a clean volume after the API has come up at least once (so migrations have run):

```
docker compose --env-file deploy/production.env down -v
docker compose --env-file deploy/production.env up --build -d
docker cp /tmp/kneo_serv_data_restore.sql \
  <db-container-id>:/tmp/kneo_serv_data_restore.sql
docker compose --env-file deploy/production.env exec -T db \
  psql -v ON_ERROR_STOP=1 -U "$POSTGRES_USER" -d "$POSTGRES_DB" \
  -f /tmp/kneo_serv_data_restore.sql
docker compose --env-file deploy/production.env restart api
```

`ON_ERROR_STOP=1` aborts the restore on the first failing INSERT so you don't end up with partial state.

SQLite — single-host installs

When `KNEO_SERV_DATABASE_URL` is unset, run state lives in `.kneo/kneo_runs.sqlite` and continuations in `.kneo/continuations/`. The service ships an online backup helper:

```
from kneo_serv.maintenance import backup_sqlite_database, restore_sqlite_database

# Online – safe while the service is running
backup_sqlite_database(
    ".kneo/kneo_runs.sqlite",
    ".kneo/backups/kneo_runs-2026-05-12.sqlite",
)

# Restore into a new location, then swap into place during a window
restore_sqlite_database(
    ".kneo/backups/kneo_runs-2026-05-12.sqlite",
    ".kneo/kneo_runs.restored.sqlite",
)
```

`backup_sqlite_database` uses SQLite's `backup()` API and is safe to run against a live database. `restore_sqlite_database` is a plain file copy — stop the service before swapping the restored file into the live path, or you'll race the writer.

Also back up `.kneo/continuations/` and any artifact paths your specs write to; these are not inside the SQLite file.

Backup frequency

There is no single recommended cadence. Tie it to your retention policy and your tolerance for re-running work:

Workload shape	Cadence
Low run volume, short retention	Daily dump, 30-day retention
Active production, multi-day retention enabled (<code>KNEO_SERV_RETENTION_*</code>)	Hourly dump, 7-day retention; daily off-site copy
Audit-heavy compliance workloads	Per-hour dump kept for the compliance window; verified test-restore monthly

The relevant env vars are in [environment.md § Retention](#). A retention policy that prunes runs after 7 days needs backups newer than 7 days, or the restore set is empty.

Verifying a restore

Backups are unproven until they have been restored. Verify on the schedule below, not after a real incident.

1. Provision a scratch host or namespace and restore the backup into it.
2. Start `kneo-serv` against the restored database.
3. Verify dependencies: `bash curl -sf http://127.0.0.1:8000/readyz | jq '.metadata.ready' # → true`
4. Verify a known run survived: `bash curl -sf "http://127.0.0.1:8000/v1/runs?limit=5" \ -H "Authorization: Bearer $OP_TOKEN" | jq '.runs[].run_id'`
5. Run the deployment smoke against the restored stack ([deployment_smoke.md](#)). It exercises `run create → fetch → cancel` and confirms checkpoints persist.
6. Verify audit events from before the backup are present: `bash curl -sf "http://127.0.0.1:8000/v1/audit-events?limit=5" \ -H "Authorization: Bearer $OP_TOKEN" | jq '.events[].event_type'`

Recommended cadence: monthly restore drill into a scratch environment, plus a restore drill immediately before any major upgrade.

Rolling back after a failed upgrade

Migrations are schema-forward and not safe to downgrade in place. If a new release misbehaves and the issue can't be patched forward:

1. **Stop the service.** Quiesce writers; the proxy can keep returning `503` from `/readyz` until step 5. `bash docker compose --env-file deploy/production.env stop api`

2. **Restore persistence** from the pre-upgrade backup using the PostgreSQL or SQLite procedure above.
3. **Re-install the previous version.** Pin the image tag or reinstall the pip package at the prior version. Update Compose / Kubernetes manifests accordingly.
4. **Restart the service.** `bash docker compose --env-file deploy/production.env start api`
5. **Verify** with `curl /readyz` and the deployment smoke ([deployment_smoke.md](#)).

Keep the pre-upgrade backup until you have verified the new version through at least one business cycle.

Disaster recovery checklist

Scenario	Recovery
Lost the host, database survived	Provision new host → install <code>kneo-serv</code> → point <code>KNEO_SERV_DATABASE_URL</code> at the surviving DB → start.
Lost the database	Provision DB → restore latest dump → start service → verify <code>/readyz</code> and a known run.
Lost host and database	Provision DB → restore latest off-site dump → provision host → start service → verify.
Corrupted checkpoints for one run	Use <code>GET /v1/runs/{run_id}/checkpoints/diff</code> to identify the bad checkpoint; cancel and re-run from the last good step. The DB itself is fine.
Restore brought back stale data, signs of mismatch	See troubleshooting.md § 2.5 for the recovery shape.

What this page does not cover

- **Performance and capacity sizing.** Covered in its own guide: [performance.md](#) — throughput, latency, store choice, and the bench harness for reproducing numbers on your own hardware.
- **The Python backup API surface.** Stays in [service_api.md § Backup and restore](#).
- **Release-team verification gates** for the GA cut. Those live in [release_checklist.md](#).

Upgrade guide

Source: <docs/user/upgrade.md>

Conventions for upgrading Kneo Agent Platform (`kneo-serv`) between releases, plus version-specific notes when a release has breaking changes.

For the release process itself (gates, tagging, artifacts), see release_checklist.md. For the supported `kneo_agent` SDK range, see sdk_alignment.md.

Versioning

Kneo Agent Platform follows semantic versioning:

- **Patch** (`0.1.0` → `0.1.1`): bug fixes; persistence schemas, route contracts, CLI commands, and env-var names do not change.
- **Minor** (`0.1.x` → `0.2.0`): additive changes. Persistence schemas may add new tables or columns with migrations; routes and CLI may add new surfaces. Existing surfaces remain available with the same shape unless the release notes call out an exception.
- **Major** (`0.x` → `1.0`): may remove or change surfaces. Read the release notes before upgrading; expect to update calling code.

The HTTP API is also versioned at the URL prefix (`/v1`); legacy unversioned routes remain available alongside `/v1`. See <design.md> § 13.

Standard upgrade procedure

1. **Read the release notes** for every minor/major version between your current and target version. Patch upgrades only need the latest patch's notes. Notes for the current release are at release_notes_0.1.0.md.
2. **Pin the target version** in your dependency manifest: `kneo-serv[deploy]==X.Y.Z`
3. **Stop traffic** to the service (or drain via a load balancer). Background runs that are queued will be reclaimed by the worker after restart; in-flight runs that complete during the drain will record normally.
4. **Back up persistence**. Follow backup_and_recovery.md (`pg_dump` for PostgreSQL, `backup_sqlite_database()` for SQLite). Keep the backup until you have verified the new version through at least one business cycle.
5. **Install the new version** in your deployment image or environment.
6. **Restart the service**. Migrations apply automatically at startup. Watch the structured log for `migration` events and any `migration_failed` errors.
7. **Verify** with `GET /readyz` and the deployment smoke script: `bash python scripts/deployment_smoke.py --base-url http://<host>:<port>`

8. Resume traffic.

If `GET /readyz` does not return 200 within a few seconds of restart, see [troubleshooting.md § 1.2](#).

Persistence migrations

Every store that has a schema (`SQLiteRunStateStore` , `PostgresRunStateStore`) tracks its schema version and applies forward-only migrations on first connection. Migrations are idempotent and never drop columns or rows on their own. The file-based stores have no schema; they tolerate older record shapes through the row decoder.

If a migration fails, the service refuses to serve requests rather than running on a partially-migrated schema. Fix the underlying cause (usually a permissions or disk-space problem), then restart.

Downgrades are not supported. Restore from backup if you need to revert.

For contributors authoring new migrations (conventions, the dialect portability rules, the test patterns), see [docs/dev/migrations.md](#) .

Spec migrations

The YAML spec format is versioned at `version: v1` . The compiler accepts older shapes through automatic normalization, but for clarity the CLI can write upgraded specs to disk:

```
kneo spec migrate legacy_agent.yaml --output migrated_agent.yaml
kneo spec migrate migrated_agent.yaml --check --json
```

Specs that pass `kneo spec validate` on the source version will continue to compile after upgrading; specs that hit deprecation warnings should be migrated proactively before a future release removes the fallback.

Signed bundles created with `kneo spec bundle sign` are tied to the signing key, not the kneo-serv version, so bundles signed before an upgrade continue to verify after as long as the signing key is unchanged.

SDK compatibility

`kneo-serv` declares a `kneo-agent` range in `pyproject.toml` . When upgrading kneo-serv, let pip resolve the matching SDK; do not pin SDK versions outside that range. The compatibility tests (`tests/test_sdk_compatibility.py`) assert the SDK surface used by the service, so a version mismatch surfaces as a test failure.

If you maintain custom runtimes or middlewares that import directly from `kneo_agent`, run those compatibility tests after upgrading and update imports in lockstep.

Configuration changes

Environment-variable names and defaults are part of the public surface. Changes are recorded in [environment.md](#) and called out in release notes:

- New variables default to behavior consistent with the previous release.
- Renamed variables retain a deprecation alias for at least one minor release; a startup warning is emitted when the alias is used.
- Removed variables are removed only at major versions.

After upgrading, diff your env file against the latest [deploy/production.env.example](#) (or `staging.env.example`) to spot any new optional variables.

CLI changes

The `kneo` CLI is regenerated each release; see [cli_reference.md](#) for the current shape. New subcommands are additive within minor releases. Subcommand behavior may change at major releases — check the release notes.

CLI profiles stored at `~/.kneo_serv/profiles.json` carry forward across releases. The profile schema is itself versioned and migrated in place.

Version-specific notes

This section grows as releases ship. Each entry should describe what changed, what action operators must take, and how to verify the upgrade.

0.1.0 — initial release

No upgrade applies; this is the first published version. See [release_notes_0.1.0.md](#) for scope, capabilities, and verified release-candidate steps.

0.2.0 — first public distribution

This is the first cut to publish a real `kneo-serv` package. 0.1.0 and 0.1.1 shipped as GitHub Release artifacts only; 0.2.0 is the first version available via `pip install kneo-serv` and `docker pull ghcr.io/kneo-agent/kneo-serv`.

Version trajectory on PyPI: 0.0.0 → 0.2.0. The `kneo-serv 0.0.0` placeholder published on 2026-05-14 reserved the distribution name; it shipped an empty importable module with no `kneo` CLI binary (no `[project.scripts]` entry). Any user who tried `pip install kneo-serv && kneo --version` during the placeholder window saw `kneo: command not found` — 0.2.0 is the first cut to install the binary. The placeholder is yanked once 0.2.0 ships; existing explicit `==0.0.0` pins still resolve, but default `pip install kneo-serv` jumps straight to 0.2.0.

Install paths: - `pip install kneo-serv` — first time this works end-to-end. - `docker pull ghcr.io/kneo-agent/kneo-serv:0.2.0` (and `:0.2`, and `:latest`) — first time the image is available without a local build.

Deployment migration for operators on 0.1.x using `compose.yaml` with the bundled `build:` context: `.` - Default flow becomes `docker compose pull && docker compose up -d` against the GHCR image. - The `build:` block stays in `compose.yaml` for contributors and the CI smoke test (`docker compose up --build`). - No required changes to `deploy/production.env` or `deploy/staging.env` from 0.1.1.

Persistence schemas: unchanged from 0.1.1. No migrations required.

Feature additions visible to operators (full per-feature detail in [release_notes_0.2.0.md](#)): - `kneo spec lint` — CI-friendly validator subcommand that exits non-zero on any warnings or errors. - Retention windows now live in `.kneo/config.yaml` under a `retention:` block, with env vars as the operator override. - Human-task expiration via `PlatformManager.prune_expired_human_tasks()` — paused runs whose human-step deadline has passed transition to a new `expired` status and emit `human.expired` audit events. - Two new reference example specs: `concurrent_review_workflow.yaml` and `group_chat_workflow.yaml`. - Docker-based local PostgreSQL integration testing via `python scripts/postgres_test.py`.

No breaking changes to spec syntax, HTTP API contracts, CLI command names, env-var names, or persistence schemas. Specs that validated under 0.1.1 continue to validate under 0.2.0.

0.2.1 — `/healthz` version and Docker `/app` permission fix

Patch release fixing two regressions discovered while smoke-testing the published 0.2.0 image. Both are bug fixes; no new features, no contract changes.

Upgrade: - `pip install -U kneo-serv` (resolves to 0.2.1). - `docker pull ghcr.io/kneo-agent/kneo-serv:0.2.1` — `:0.2` and `:latest` now resolve to the 0.2.1 digest.

What was broken in 0.2.0: - `GET /healthz` returned `"version":"0.1.0"` from the 0.2.0 image because `HealthResponse.version` was a hardcoded string literal. 0.2.1 resolves the field dynamically via `importlib.metadata.version("kneo-serv")`. - Plain `docker run -p 8000:8000 ghcr.io/kneo-agent/kneo-serv:0.2.0` crashed on startup with `PermissionError: [Errno 13] Permission denied: '.kneo'` because `/app` was root-owned but the container drops to the non-root `kneo` user before creating the SQLite-fallback path. 0.2.1 adds `chown -R kneo:kneo /app` to

the install layer. The Docker Compose deployment path was unaffected (it pins `KNEO_SERV_DATABASE_URL` to PostgreSQL).

Persistence schemas: unchanged from 0.2.0. No migrations required.

No breaking changes to spec syntax, HTTP API contracts, CLI command names, env-var names, or persistence schemas.

0.2.2 — FastAPI `info.version` fix + post-0.2.0 docs sweep

Patch release fixing one regression in the same family as 0.2.1 plus a documentation sweep. No feature changes, no contract changes, no schema changes.

Upgrade: - `pip install -U kneo-serv` (resolves to 0.2.2). - `docker pull ghcr.io/kneo-agent/kneo-serv:0.2.2` — `:0.2` and `:latest` now resolve to the 0.2.2 digest.

What was broken in 0.2.1: - GET `/openapi.json` returned `info.version: "0.1.0"` from the 0.2.1 image because the FastAPI app constructor in `kneo_serv/service/app.py` still pinned a hardcoded literal. The 0.2.1 cut fixed `HealthResponse.version` but missed this parallel occurrence. 0.2.2 resolves both via the same `importlib.metadata.version("kneo-serv")` helper, called at app-construction time.

Documentation: - Forward-looking plan docs and "as of 0.1.0" framing in user/dev docs swept to match the 0.2.x shipped reality. No content lost — historical files (CHANGELOG entries, shipped release notes, `TOD0-0.2.0.md`, ADRs) are unchanged.

Persistence schemas: unchanged from 0.2.1. No migrations required.

0.3.0

Next additive minor on the 0.2.x line. No breaking changes to spec syntax, HTTP API contracts, CLI command names, env-var names, or persistence schemas. Full narrative in [release_notes_0.3.0.md](#).

Upgrade: - `pip install -U kneo-serv` (resolves to 0.3.0). - `docker pull ghcr.io/kneo-agent/kneo-serv:0.3.0` — `:0.3` and `:latest` now resolve to the 0.3.0 digest. The image is now signed (cosign keyless via Sigstore) and ships with a CycloneDX SBOM attestation; verification commands are in [supply_chain_review.md § Verification commands](#).

SDK floor bump: - The `kneo-agent` SDK floor moves from `>=1.1.1` to `>=1.2.0`. Pip auto-resolves on `pip install -U kneo-serv`, but operators pinning the SDK separately (e.g. via a constraints file or a monorepo lockfile) must ensure their install is on `1.2.0` or newer. The compat test suite passed against `kneo-agent 1.2.0` throughout the 0.2.x line; the floor was kept low to avoid forcing 0.1.x users to upgrade. 0.3.0 is the natural inflection point to lift it.

New `timed_out` lifecycle status: - Runs that hit their run-level deadline transition to a new terminal `timed_out` status (alongside `completed`, `failed`, `cancelled`, `expired`). Operator tooling that switches on `state.status` should accept it as terminal — e.g. dashboards, alerting rules, retention sweeps (which the platform's own `RetentionPolicy.run_statuses` already includes). - The `error.type` field on a timed-out run is `run_timed_out`, distinct from `human_task_expired` (which the existing `expired` status uses).

New runtime surfaces: - `start_run_from_spec(..., timeout_seconds=N)` and `run_from_spec(..., timeout_seconds=N)` accept an optional wall-clock deadline. Operator-callable `PlatformManager.prune_timed_out_runs()` walks runs and force-cancels those past their deadline. Same operator-cron pattern as `prune_retention()` and `prune_expired_human_tasks()` — no built-in scheduler. - The human-task `on_timeout: continue` and `on_timeout: escalate` literals are now wired in the runtime (they were accepted by the spec but silently treated as `fail` in 0.2.x). Operators with specs that declared these literals will see the documented behaviour for the first time. Audit consumers should expect new event types: `human.continued`, `human.continue_failed`, `human.escalated`, `run.timed_out`. - New route `GET /v1/runs/{run_id}/policy-report` returns the spec policy report for a stored run, no spec bundle required client-side. Auth: `specs:read` scope (same as the existing `POST /v1/specs/policy-report`).

New observability surfaces: - Three new platform-side OpenTelemetry spans (`kneo.queue.dispatch`, `kneo.worker.lease`, `kneo.continuation.lock`) join the SDK's agent-boundary spans when `KNEO_SERV_OTEL_ENABLED=true`. Pre-existing OTel pipelines pick them up automatically once telemetry is enabled — no extra configuration required. See [observability.md § Platform-side spans](#).

Persistence schemas: unchanged. The new `RunState.deadline_at` and `Checkpoint.iteration` fields default to `None` and `1` respectively in the dataclass, so existing rows round-trip cleanly through the JSON-payload SQLite / PostgreSQL stores.

0.4.0

Next additive minor on the 0.3.x line. **No breaking changes** to spec syntax, HTTP API contracts, CLI command names, env-var names, or persistence schemas. Specs that validated under 0.3.x continue to validate under 0.4.0. The cut is a **docs + tooling release** — runtime semantics are identical to 0.3.0. Full narrative in [release notes 0.4.0.md](#).

Upgrade: - `pip install -U kneo-serv` (resolves to 0.4.0). - `docker pull ghcr.io/kneo-agent/kneo-serv:0.4.0` — `:0.4` and `:latest` now resolve to the 0.4.0 digest. Image continues to be signed (cosign keyless via Sigstore) and ships with a CycloneDX SBOM attestation; the 0.4.0 cut adds a Trivy CVE scan report attached to the GitHub Release. Verification commands are in [supply_chain_review.md § Verification commands](#).

SDK floor: unchanged. The `kneo-agent` floor stays at `>=1.2.0` — same as 0.3.0. No operator action required for operators pinning the SDK separately.

New auto-generated API reference: the docs site at `kneo-agent.github.io/kneo-serv/` gains a new top-level **API Reference** nav section with 17 pages (16 subpackages + `sdk`), rendered at build time by `mkdocstrings` from the Python docstrings. Operator surface unchanged — the API ref is a **developer lookup surface**, not a runtime change. See [docs/api/README.md](https://kneo-agent.github.io/kneo-serv/docs/api/README.md) for the index.

Image vulnerability scanning (Trivy): the release pipeline now scans the pushed GHCR image with Trivy under the CVSS≥7 policy (HIGH/CRITICAL findings block the publish step). On every release-tag build, the JSON scan report is attached to the GitHub Release as the `trivy-report-<version>` artifact, 90-day retention. Deployers can re-run the scan locally with `trivy image ghcr.io/kneo-agent/kneo-serv:<tag>`; full policy + escape hatch documented in [security_hardening.md § Image vulnerability scanning](#).

Developer-facing changes (no operator surface impact): - Ratcheting ruff D-rule gate (D100 / D101 / D102) now enforced project-wide for `kneo_serv/**/*.py`. New public classes / methods without docstrings fail CI. Forks adding code should follow the Google docstring convention; the chain-reference files are [security/secrets.py](#) and [platform/manager.py](#). - Full mypy strict coverage across `kneo_serv/`. The `[[tool.mypy.overrides]]` block in [pyproject.toml](#) now covers every public module. Forks that subclass or extend public types should expect `disallow_untyped_defs + warn_return_any + strict_equality`. - `mkdocstrings[python]>=0.27` added to the `docs` optional-dep block. Operators using `pip install kneo-serv` (without `[docs]`) are unaffected — the dep is build-time only for the rendered site.

New 0.3.0-feature worked examples: - [examples.md](#) picked up a *Timeout branches* subsection on the `human_approval_workflow.yaml` entry covering the `on_timeout: fail/continue/escalate` literals (all wired since 0.3.0). - New [examples/run_with_timeout.py](#) walks through `start_run_from_spec(..., timeout_seconds=N) + prune_timed_out_runs()`. Companion to the human-task timeout example above.

Persistence schemas: unchanged. No new fields, no migrations.

0.5.0

Next additive minor on the 0.4.x line. **No breaking changes** to spec syntax, HTTP API contracts, CLI command names, env-var names, or persistence schemas. Specs that validated under 0.4.x continue to validate under 0.5.0.

Upgrade: - `pip install -U kneo-serv` (resolves to 0.5.0). - `docker pull ghcr.io/kneo-agent/kneo-serv:0.5.0` — `:0.5` and `:latest` resolve to the 0.5.0 digest; image signing, SBOM attestation, and the Trivy scan gate are unchanged from 0.4.0.

SDK floor: unchanged. The `kneo-agent` floor stays at `>=1.2.0`.

Bug fix — checkpoint-callback metadata now survives on the final run record: in 0.4.x and earlier, `execute_run` and `continue_run` saved a stale in-memory `RunState` at the end of a run, overwriting the `step_iteration_counts` and `execution_context` that the checkpoint callback had written to `RunState.metadata` during the run. The values survived on the *checkpoints* but were missing from the *run record*. 0.5.0 re-reads the persisted run before the final save. **Operator impact:** if you read per-step iteration metadata off the final `RunState` (via `GET /v1/runs/{id}` metadata or the `PlatformManager` API) and worked around its absence, that metadata is now present. Checkpoint contents are unchanged. No migration or data backfill — the fix only affects runs executed under 0.5.0 onward.

New performance and capacity guide: [performance.md](#) documents throughput, latency, the SQLite-vs-PostgreSQL capacity trade-off, and the tuning knobs, with a reproducible bench harness ([scripts/bench](#)). No runtime change — guidance and tooling only.

Single-team on-prem operability (all additive; defaults preserve 0.4.x behaviour):

- **Worker concurrency.** `KNEO_SERV_WORKER_CONCURRENCY` (default `1`) runs a pool of in-process worker threads; `KNEO_SERV_WORKER_LEASE_SECONDS` (default `300`) sets the queue lease. Default `1` reproduces the prior single-worker behaviour. Sizing guidance in [performance.md](#).
- **Prometheus /metrics.** New unauthenticated `GET /metrics` (root path only), opt-out via `KNEO_SERV_METRICS_ENABLED=false`.
- **Operator action:** restrict it to your monitoring network (reverse proxy or the env flag). See [observability.md § Prometheus /metrics](#).
- **Overload backpressure.** `KNEO_SERV_MAX_QUEUE_DEPTH` (default `0` = unlimited, i.e. unchanged). When set, `POST /v1/runs` (async) returns `503` with `Retry-After: 5` once the queue is full — make async callers retry on `503`.
- **Poison-run dead-letter.** `KNEO_SERV_QUEUE_MAX_ATTEMPTS` (default `5`) fails a run re-leased past the cap with a `dead_letter` error + `run.dead_lettered` audit event.
- **Behaviour change:** before 0.5.0 a run that repeatedly crashed its worker was re-leased indefinitely; it is now dead-lettered after 5 attempts. Set `0` to restore unbounded retries.
- **Graceful drain.** On `SIGTERM` the worker pool finishes in-flight runs and stops claiming new ones, so container rollouts no longer interrupt a run.

Persistence schemas: unchanged. No new fields, no migrations.

0.6.0

Multi-process PostgreSQL hardening + uptake of the Kneo Agent SDK 2.x line. **Additive persistence migrations only** (a new checkpoint uniqueness index); no breaking spec-syntax, CLI, or env-var-removal changes. Two intentional *observable* changes (error codes, `RunConfig` defaults) are called out below.

Upgrade: - `pip install -U kneo-serv` (resolves to `0.6.0`). - `docker pull ghcr.io/kneo-agent/kneo-serv:0.6.0` — `:0.6` and `:latest` resolve to the 0.6.0 digest; image signing, SBOM attestation, and the Trivy scan gate are unchanged.

SDK floor bumped to `kneo-agent>=2.2.0,<3.0.0` (from `>=1.2.0,<2.0.0`). The service installs this for you; if you pin `kneo-agent` yourself, move your pin onto the 2.x line. The bump picks up SDK

security fixes (MCP cross-origin redirect refusal + URL-scheme validation; broader secret redaction). Two SDK 2.x behaviour deltas a deployer may notice: - **The SDK no longer auto-retries tool calls.** kneo-serv's own retry knobs (`KNEO_SERV_PROVIDER_RETRIES` , `KNEO_SERV_MCP_RETRIES` , ...) are unchanged and still apply; only the SDK's internal per-tool retry default flipped off. - **Spec/agent with_defaults for temperature / max_iterations now actually apply** — see the `RunConfig` change below.

Observable change — error-code remap (action may be required). The public `error` field in error responses is now a stable, snake_case code decoupled from internal Python class names:

Status	Old error	New error
404	<code>KeyError</code>	<code>not_found</code>
400	<code>ValueError</code> / <code>FileNotFoundError</code>	<code>invalid_request</code>
500	<code><ExceptionClassName></code> (+ raw message)	<code>internal_error</code> (generic message)

`queue_full` , `resource_locked` , `unauthorized` , `forbidden` , and the idempotency codes are unchanged. If a client matched on the old class-name codes (`KeyError` , `ValueError` , ...), update it to the new codes. The error envelope — `{"detail": {"error": "...", "message": "..."} }` — is otherwise unchanged and is now published in the OpenAPI schema as `ErrorResponse` / `ErrorDetail` . 500 responses no longer echo the exception message, and `/readyz` probe failures no longer leak the underlying error detail (both are logged server-side instead).

Observable change — RunConfig defaults now merge. Before 0.6.0, a run that didn't specify `max_iterations` / `temperature` had them force-set to `10` / `0.7` , silently overriding a spec author's `with_defaults(...)` . 0.6.0 leaves them unspecified so the agent/skill defaults apply (SDK 2.x merge semantics). If a spec set `temperature: 0.2` (or a custom `max_iterations`) and you relied on the run *ignoring* it, the spec value now takes effect. To force a value regardless of the spec, set it explicitly on the run config. A malformed `temperature` (non-numeric / `bool` / `NaN` / `inf`) now returns `400 invalid_request` instead of a 500.

New operator knobs (all additive; defaults preserve 0.5.x behaviour): -

KNEO_SERV_RETENTION_AUDIT_DAYS (and project-config `retention.audit_days`) — prune audit events older than N days. The audit table is otherwise unbounded; set this on long-lived deployments. - **KNEO_SERV_SHUTDOWN_TIMEOUT_SECONDS** (default `30`) — how long SIGTERM shutdown waits for in-flight runs to finish. A run still executing past the timeout is interrupted by process exit but stays claimed and is re-released / retried (not lost); set this **and** your orchestrator's termination grace period $\geq$ your longest run step to drain without a restart. - **Token-usage metrics.** `/metrics` now exposes `kneo_tokens_input_total` , `kneo_tokens_output_total` , and `kneo_tokens_total` counters; usage is also on the run record and `run.created` audit metadata when the runtime reports it. -

Idempotency in-progress. A duplicate `POST` arriving while the first same-key request is still in flight now returns `409 idempotency_key_in_progress` (previously the two could race). Treat `409` as "retry shortly".

Persistence schemas: one **additive** migration — a `UNIQUE(run_id, sequence)` index on `checkpoints` (migration v3), which de-duplicates any pre-existing duplicate `(run_id, sequence)` rows on first start. PostgreSQL queue/lease timestamp columns are widened `REAL` → `DOUBLE PRECISION` in place (a precision fix). No data backfill or operator action required; both apply automatically on the first start under 0.6.0.

0.7.0

Finishing the 0.6.0 lease-liveness story plus an on-prem operability cluster. **No breaking changes, no persistence migration, no SDK-floor change** — the schema version is unchanged from 0.6.0 and the SDK floor stays `kneo-agent>=2.2.0,<3.0.0`. Two *behaviour* notes below are worth reading before you upgrade; everything else is additive and default-off / default-unset.

Upgrade: - `pip install -U kneo-serv` (resolves to 0.7.0). - `docker pull ghcr.io/kneo-agent/kneo-serv:0.7.0` — `:0.7` and `:latest` resolve to the 0.7.0 digest. The release pipeline now runs an **in-pipeline cosign verify self-check** (`cosign verify` + `verify-attestation` against the pushed digest); image signing, SBOM attestation, and the Trivy gate are otherwise unchanged.

Behaviour note — `worker_lease_seconds` is now a liveness window, not a run-time cap. A worker now renews its queue lease for the life of a run (a heartbeat renewing at $\sim \text{worker_lease_seconds} / 3$), so a healthy long run never lets its lease lapse and get reclaimed mid-flight. The lease therefore no longer bounds how long a run may take — it bounds how long a *crashed* worker's run stays unreclaimable. If you raised `worker_lease_seconds` in 0.5.x/0.6.x to "fit" your longest run, you can lower it back toward your crash-detection latency. No action is required; the default is unchanged and shorter leases are now safe.

Behaviour note — the per-run token ceiling is a post-run boundary check, not a mid-flight kill. A run can be capped at a maximum input+output token budget via the SDK's `TokenBudgetMiddleware`, configured per agent with `model.token_budget` in the spec (a **positive** integer — a non-positive value is rejected at spec validation, not silently ignored) or deployment-wide with `KNEO_SERV_TOKEN_BUDGET` (the spec field wins). The middleware checks reported usage **after** each run and then raises `TokenBudgetExceeded`, surfaced as `400 token_budget_exceeded`. Consequences: - A run that overshoots within a single step finishes that step before failing — size the ceiling as a spend **backstop**, not a precise hard stop. - `on_missing="ignore"`: a runtime that doesn't report `metadata["usage"]` never spuriously fails the ceiling. Unset (the default) means no ceiling.

New operator knobs (all additive; defaults preserve 0.6.x behaviour): -

KNEO_SERV_AUDIT_EXPORT_ENABLED — when set, every persisted (already redacted) audit event is also emitted as a JSON line on the dedicated `kneo_serv.audit` logger, from the single `record_audit_event` chokepoint. Attach a logging handler to forward to a file / syslog / SIEM. Off by

default; export failures never break the run path. - `KNEO_SERV_TOKEN_BUDGET` — deployment-wide per-run token ceiling (see the behaviour note above). A spec's `model.token_budget` overrides it. -

Local / self-hosted LLM endpoints. The native (`openai`) runtime now reads

`model.extra.base_url` and an API key — `model.extra.api_key_ref` resolved through the `SecretResolver`, or a literal `api_key` escape hatch — and threads them into the OpenAI-compatible client, so a spec can target Ollama / vLLM / llama.cpp / LocalAI. Unset fields preserve the hosted-OpenAI default; a literal `api_key` is redacted from audit / list surfaces.

New spec fields (optional, additive — old specs are unaffected): - **Human-request taxonomy.**

`components.humans.*` accepts `request_type` (`approval` / `review` / `correction` / `selection` / `freeform`), `options`, `default_option`, `context`, and `response_role`. `validate_semantics` rejects a `default_option` outside `options` and a `selection` without `options`. When `response_role` is set, the reviewer's reply folds into the resumed run's message thread with that role. `GET /v1/human-tasks/{id}` now also returns the paused run's **redacted messages thread** alongside the pending `request` (same auth scope; no new route). A client that only reads the existing `request` field is unaffected.

Persistence schemas: no migration. The store schema version is unchanged from 0.6.0;

`RunStateStore` gains a `schema_version` / `close` Protocol surface (behaviourally a no-op on the schema-less stores), but no on-disk change applies on upgrade.

0.8.0

Declarative spec parity along the tools / MCP / skills axis. **No breaking changes, no persistence migration, no SDK-floor change** — the schema version is unchanged and the SDK floor stays `kneo-agent>=2.2.0,<3.0.0`. One *behaviour* note below is worth reading before you upgrade; everything else is additive and default-unset.

Upgrade: - `pip install -U kneo-serv` (resolves to 0.8.0). - `docker pull ghcr.io/kneo-agent/kneo-serv:0.8.0` — `:0.8` and `:latest` resolve to the 0.8.0 digest. Image signing, SBOM attestation, and the Trivy gate are unchanged from 0.7.0.

Behaviour note — `overlays` is no longer silently ignored. `POST /v1/runs` (sync + async) and the `/v1/specs/run` / `/compile` / `/validate` / `/policy-report` routes accepted an `overlays` list but dropped it without applying it. From 0.8.0 the overlays are threaded through compile/run, persisted in run metadata, and replayed on resume. If any stored client request or automation passes `overlays`, audit it before upgrading — those overlays now actually change the compiled spec. `overrides` / `strict` are likewise now honored on the `/specs/*` routes that previously dropped them.

Trust note — `spec_path` and `overlays` are filesystem-trusted inputs. Both name paths the *server* reads at compile time. Grant `runs:write` / `specs:read` -scoped keys to callers you trust with that read surface, and see [security_hardening.md](#) for the posture before exposing these fields to semi-trusted callers.

New spec surface (all optional, additive — old specs are unaffected):

- **Declarative MCP transports.** A top-level `mcp_servers` block (`transport: stdio | http | sse` , with `command / args / env / cwd` or `url / sse_url / message_url / headers / timeout` , plus `max_response_bytes / sse_read_timeout` knobs and `verify / ca_bundle / client_cert / client_key` TLS fields) and a `tool.mcp = {server: <name>, name?: <remote_tool>}` reference. Construction happens at build time; the connection is lazy on first tool call, so the spec compiles offline. **Prefer `client_key_ref`** (resolved via the `SecretResolver`) over inline `client_key` — the inline spec is persisted unredacted into run metadata, and the TLS field names are redaction terms only on audit/list surfaces. `verify: false` draws a validation warning.
- **Agent-as-tool.** `tool.agent: <name>` backs a tool with another declared agent. A tool must be backed by exactly one of `implementation / mcp / agent` — a tool with none is now a validation error (`E_TOOL_NO_BACKING`) instead of being silently dropped at build.
- **Workflow-as-agent.** `agent.as_agent: <workflow>` backs an agent with a declared workflow; only `name / description / system_prompt` are legal alongside it. Cyclic or dangling references across all of these fail at `/specs/validate` (`E_BUILD_CYCLE` etc.), not at runtime.

New API surface (additive):

- **GET `/v1/skills`** — read-only catalog of declared + default discoverable skills; `specs:read` scope, standard pagination, no side effects.
- **RunCreateRequest.skills** — per-request `{add, disable}` skills overlay. `add` only enables skills already declared in the spec; out-of-scope overlays are rejected; every overlay is audited (`run.skills_overlay`) and preserved across resume.
- **GET `/v1/human-tasks?status=pending|escalated`** — a real filter now; an unknown value returns 422 where it was previously a silent no-op.
- **POST `/runs/{id}/continue`** accepts an `Idempotency-Key` and replays the stored response on retry; concurrent `/continue` calls are serialized under a per-run lock. `POST /v1/specs/run` now holds the same idempotency lock as `/runs` (409 `idempotency_key_in_progress` on contention).
- **Invalid specs on sync POST `/v1/runs` return 400** with diagnostics where they previously surfaced as an opaque 500. 413 (`payload_too_large`) is now published in the OpenAPI error responses.

Persistence schemas: no migration. No new persisted fields; rollback to 0.7.x after running 0.8.0 is persistence-safe (the new request/spec fields are request-scoped or compile-scoped only).

Rolling back

Schema-forward migrations make in-place downgrade unsafe; the only supported rollback path is **restore from the pre-upgrade backup**, then re-install the previous version.

For the full step-by-step procedure — stop, restore, re-install, restart, verify with the deployment smoke — see [backup_and_recovery.md § Rolling back after a failed upgrade](#).

Keep the pre-upgrade backup until you have verified the new version through at least one business cycle.

Reporting upgrade issues

Capture the same context listed in [troubleshooting.md § What to capture before opening a bug](#), plus:

- Source version (`pip show kneo-serv` before the upgrade).
- Target version (after the upgrade).
- Migration log lines from the first start on the new version.
- The exact env file or compose `.env` (with secrets redacted).

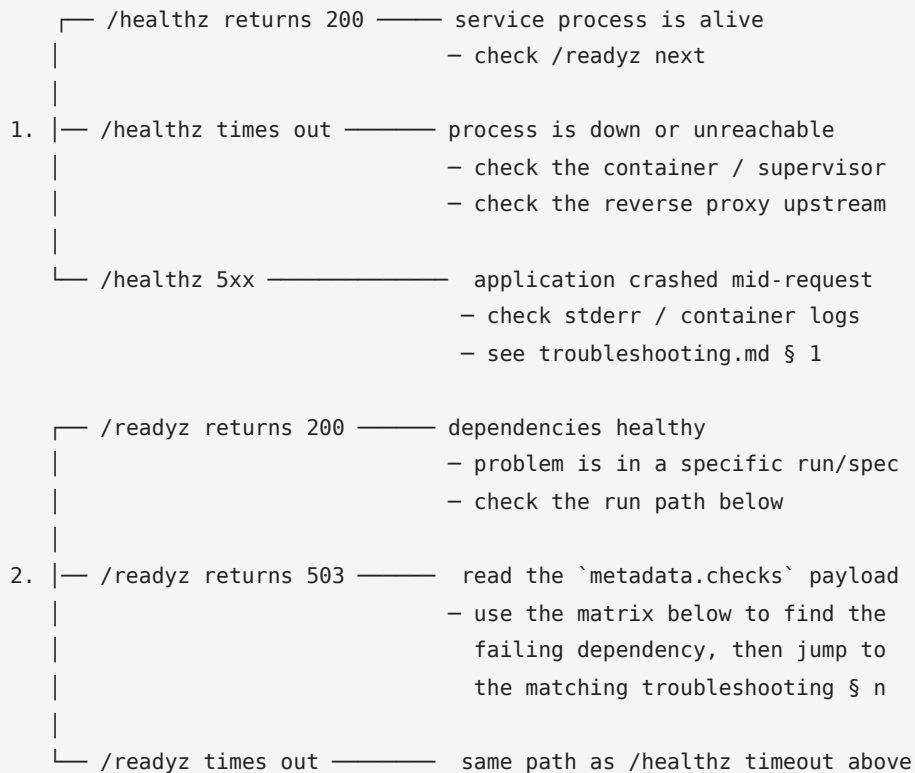
Incident response

Source: docs/user/incident_response.md

On-call entry point for `kneo-serv`. This page is the triage tree — "the service looks wrong, where do I look first?" The symptom-by-symptom deep dive lives in <troubleshooting.md>; this page sends you there.

For backup and rollback procedures, see backup_and_recovery.md. For the API definition of the health endpoints, see service_api.md § [Health checks](#).

Triage tree



`/healthz` and `/readyz` are unauthenticated by design — you can probe them from anywhere you can reach the service port.

```
curl -sf http://<host>:<port>/healthz | jq
curl -sf http://<host>:<port>/readyz | jq '.metadata.checks'
```

`/readyz` failure matrix

`/readyz` runs eight checks; when any fails, the response is `503` with `{"error": "not_ready", "metadata": {"checks": {...}}}`. The keys you will see and what each means:

([kneo_serv/service/routes_health.py](#))

Check key	What it probes	Common failure	Where to go
<code>api</code>	API wiring sentinel	Never fails on its own	If it does, the manager isn't configured — § 1.3
<code>run_state_store</code>	<code>manager.run_state_store.list_runs(limit=1)</code> succeeds	DB unreachable, schema missing, credentials wrong	§ 2.1 / § 2.2
<code>continuation_store</code>	<code>manager.continuation_store.list()</code> succeeds	File path missing, permissions wrong, DB issue	§ 2
<code>queue</code>	<code>list_queued_runs(status=...)</code> returns for <code>queued</code> / <code>running</code> / <code>failed</code>	Queue table missing or DB stall	§ 2 , § 5.1
<code>runtime_registry</code>	Number of registered runtimes (declared via factories)	Empty registry — service started without runtimes	extending.md for runtime registration
<code>tool_registry</code>	Number of registered tools	Empty registry — service started without tools	extending.md
<code>providers</code>	Secrets named in <code>KNEO_SERV_HEALTH_PROVIDERS</code> resolve	Provider env var missing or empty	§ 3.2
<code>mcp</code>	Secrets named in <code>KNEO_SERV_HEALTH_MCP_SECRETS</code> resolve	MCP secret missing	§ 3.2

The payload includes per-check `error` (exception class) and `message` for failed checks, so you usually don't have to guess which subsystem is at fault — copy the message into the relevant troubleshooting section.

Common production incidents

If `/healthz` and `/readyz` are both green but the service is "wrong":

Symptom	First check	Deep dive
All requests return <code>401</code>	<code>Authorization</code> / <code>X-Kneo-Api-Key</code> header is present and valid	§ 4.1 , § 4.3
Specific consumer returns <code>403</code>	The key's role/scope covers the route	§ 4.2 , security_hardening.md § 2
Async runs stuck in <code>queued</code>	Worker process is up; queue table reachable	§ 5.1
Runs hang mid-workflow	The step's tool or provider call is timing out	§ 5.3
<code>409 idempotency_key_conflict</code>	Caller is reusing a key with a different payload	§ 5.4
Tool reports <code>MissingSecretError</code>	Provider secret env var is set on the service host	§ 3.1
Logs missing <code>request_id</code>	You are reading the right logger (<code>kneo_serv.service</code>), not raw <code>uvicorn</code>	§ 7.1 , observability.md
OpenTelemetry exporter silent	<code>KNEO_SERV_OTEL_ENABLED=true</code> and <code>[telemetry]</code> extra installed	§ 7.2 , observability.md
Human task <code>409 resource_locked</code>	Another resume is in flight for the same continuation	§ 8.1
Restored backup but state looks stale or mismatched	Stop, re-verify the dump source, follow the recovery shape	§ 2.5 , backup_and_recovery.md

What to capture before escalating

When the runbook doesn't have an entry that fits, capture this context before paging the on-call developer. It is the same set [troubleshooting.md](#) asks for in a bug report:

- Service version and commit (`pip show kneo-serv` or the image tag).
- Environment context (`uname -a` , Python version, Postgres version).
- The `request_id` and `run_id` of an affected request.

- `GET /readyz` body, even when it returns `200`.
- For run-shaped problems: `GET /v1/runs/{run_id}`, `GET /v1/runs/{run_id}/trace`, and `GET /v1/runs/{run_id}/checkpoints` (redacted output is fine to share).
- For spec-shaped problems: the output of `kneo spec validate <path> --json`.
- Recent audit events that mention the affected resource: `GET /v1/audit-events?run_id=<id>`.

When to roll back

If the incident started immediately after a deploy and isn't covered by the matrix above:

1. Confirm the deploy is the trigger — diff the running version against the previous version, check the time correlation with the first 503/5xx.
2. If yes, follow [backup_and_recovery.md § Rolling back after a failed upgrade](#).

Rolling back when the trigger isn't the deploy throws away forward progress. Diagnose first.

What this page does not cover

- **Severity definitions and paging policy.** Owned by your on-call rota, not by `kneo-serv`.
- **Post-incident review template.** Out of scope.
- **Failure modes during a `kneo-serv` release itself** — those are in [release_checklist.md](#) and [troubleshooting.md § 9](#).

Troubleshooting

Source: <docs/user/troubleshooting.md>

An operator-facing runbook indexed by symptom. Each entry lists the symptom, how to confirm the root cause, and the fix; cross-references point at the authoritative configuration doc when one exists.

If you are responding to a live incident and don't yet have a symptom, start at [incident_response.md](#) — it walks `/healthz` → `/readyz` → the right section here. This page is the symptom-indexed deep dive.

When you're not sure where to start, check `GET /readyz` ([§ 1.2](#)) — it exposes the per-dependency status the service uses internally, and most "service is unhealthy" tickets resolve to one of its check entries.

1. Service won't start or won't accept traffic

1.1 `RuntimeError: KNEO service auth is enabled but no API keys are configured`

The service refuses to start when auth is enabled without keys. [service/auth.py](#)

- Confirm: check the startup log for the message above.
- Fix: set `KNEO_SERV_API_KEYS` (entries are `name:key:role_or_scope[,role_or_scope]`, semicolon-separated) and/or `KNEO_SERV_ADMIN_API_KEY`. To run without auth, set `KNEO_SERV_AUTH_ENABLED=false` (only for local dev).
- Reference: [environment.md § Service Auth](#), [production_readiness_review.md § Role Boundary Review](#).

1.2 `/readyz` returns 503

`GET /readyz` returns `{"error": "not_ready", "metadata": {"checks": {...}}}` when any dependency check fails. [service/routes_health.py](#)

- Confirm: `curl -sf http://<host>:<port>/readyz | jq`. Each per-dependency entry has `ok: false` plus `error` and `message` for failed checks.
- Fix: the per-check failure matrix (which check maps to which recovery action) lives in [incident_response.md § /readyz failure matrix](#). In summary: store failures are covered by §2; provider/MCP secret failures by §3.

1.3 `RuntimeError: PlatformManager has not been configured`

The default `app` factory configures the platform manager automatically. This error appears when you pass `configure_default_manager=False` to `create_app()` and never call `set_platform_manager()` before serving. [service/dependencies.py](#)

- Fix: either drop the override, or call `kneo_serv.service.dependencies.set_platform_manager(...)` before the first request.

1.4 `RuntimeError: Invalid KNEO_SERV_API_KEYS` entry

The format is `name:key:role_or_scope[,role_or_scope]` per entry, separated by semicolons. Whitespace inside entries is trimmed; missing colons trigger this error. [service/auth.py](#)

- Fix: re-render `KNEO_SERV_API_KEYS`. Examples:
- `operator:OP_TOKEN:operator;reviewer:REV_TOKEN:reviewer`
- `svc:SVC_TOKEN:runs:write,human:read,human:write`
- Reference: [environment.md § Service Auth](#), [production_readiness_review.md § Route Scope Matrix](#).

2. Persistence and store failures

2.1 PostgreSQL DSN configured but service falls back to SQLite

The service uses PostgreSQL only when `KNEO_SERV_DATABASE_URL` is set **and** the `[postgres]` or `[deploy]` extra is installed. See [service/factory.py](#).

- Confirm: in a dev shell, `python -c "import psycopg; print(psycopg.__version__)"`.
- Fix: install `kneo-serv[deploy]` (Docker image already does this), or `kneo-serv[postgres]` if you don't need telemetry.

2.2 `psycopg.OperationalError` on startup or first request

The DSN can't connect. Common causes: wrong host, missing TLS, wrong credentials, database not yet created.

- Confirm: `psql "$KNEO_SERV_DATABASE_URL" -c '\dt'` from the same network context as the service.
- Fix: correct the DSN, ensure the database exists, and confirm the user has privileges. `KNEO_SERV_DATABASE_URL` must be a libpq-style URL.

2.3 SQLite database is locked errors

Concurrent writes to a single SQLite file can collide. The default service worker is single-threaded per process; this typically appears when running multiple service processes against the same SQLite file.

- Fix: switch to PostgreSQL (set `KNEO_SERV_DATABASE_URL`). Multi-process SQLite is not a supported deployment topology.

2.4 Schema migration appears to have run but old data is missing

Migrations are idempotent and version-tracked per store; they don't drop data. If rows look missing after an upgrade, check whether you actually upgraded the same database the service is reading.

- Confirm: compare `KNEO_SERV_DATABASE_URL` (or SQLite path) between the upgrade context and the running service. A stale state file at `.kneo/kneo_runs.sqlite` is a common cause.

2.5 Backup/restore mismatch

`kneo_serv.maintenance.backup_sqlite_database()` produces a file copy that restore expects to find on the same SQLite version line. Restoring across incompatible SQLite versions can fail.

- Fix: align `sqlite3` versions, or migrate to PostgreSQL where backup goes through `pg_dump` / `pg_restore`. See [staging_release_runbook.md](#) and [release_checklist.md](#) for the seeded recovery drill.

3. Secrets, credentials, and provider integration

3.1 MissingSecretError on agent run

Provider keys, MCP credentials, and runtime settings are resolved through env-var references in project config; raw values are never stored. [security/secrets.py](#)

- Confirm: `kneo config secrets --json` lists which references exist and whether each resolves. The endpoint `GET /security/credentials` exposes the same view (requires `credentials:read`).
- Fix: export the env var named in the error. Set `KNEO_SERV_REQUIRE_PROVIDER_SECRETS=true` to fail fast at startup instead of at first run.

3.2 GET /readyz reports missing provider/MCP secrets

Readiness reports the secrets named in `KNEO_SERV_HEALTH_PROVIDERS` and `KNEO_SERV_HEALTH_MCP_SECRETS`. These are operator-curated allowlists, so expect 503 if you list a secret that isn't actually exported.

- Fix: trim the list to secrets you actually use, or export the missing one.

3.3 kneo spec bundle verify fails

Bundle verification requires `KNEO_SERV_SPEC_SIGNING_KEY` to match the key used to sign. Bundles signed with a different key (or unsigned) fail verification.

- Fix: rotate the signing key consistently across signing and verifying hosts. The key is HMAC-only; do not commit it.

4. Authentication and authorization

4.1 401 Unauthorized – A valid Kneo service API key is required

The route requires auth and the request didn't carry a valid token. [service/auth.py](#)

- Confirm: send `Authorization: Bearer <key>` or `X-Kneo-API-Key: <key>`.
- Fix: use one of the configured keys. The CLI service client reads `KNEO_SERV_API_KEY`. For multi-environment workflows use CLI profiles (`kneo config profile use ...`).

4.2 403 Forbidden – Missing required scope: <scope>

The token authenticated but the principal does not hold the scope the route requires.

[service/auth.py](#)

- Confirm: the scope in the error body tells you exactly what is missing.
- Fix: assign the principal a role that includes the scope, or add the scope explicitly in `KNEO_SERV_API_KEYS`. See the route ↔ scope matrix in [production_readiness_review.md](#).
- Common gotchas:
 - `POST /specs/run` requires `runs:write`, not `specs:read`.
 - Reviewer cannot create runs or change policies.
 - Service role cannot mutate environment policies (only operator/admin).

4.3 Health endpoints work, all other routes 401

`/healthz`, `/livez`, and `/readyz` are intentionally unauthenticated for load-balancer probes. Everything else is gated by the auth dependency. This is by design; see [production_readiness_review.md § Route Scope Matrix](#).

5. Run lifecycle problems

5.1 Async runs sit in `queued` and never progress

The platform worker is started by `create_default_platform_manager()` in [service/factory.py](#). If a custom embedding skips `manager.start_worker()`, queued runs never drain.

- Confirm: `GET /runs?status=queued` shows queued items; `GET /readyz` shows the queue dependency as ok; the service log has no "worker" lines.
- Fix: ensure `start_worker()` is called in the host process after constructing `PlatformManager` directly, or use the default factory.

5.2 Cancelled run still finishes as succeeded

Cancellation is cooperative through `CancellationToken` and propagates only at unit-of-work boundaries. A step that completes between the cancel request and the next checkpoint will record its result, but `RunState` remains `cancelled` — the platform does not overwrite cancelled status with completed results.

- Confirm: `GET /runs/{run_id}` should still report `status: cancelled` even if the last checkpoint shows completion of a step.
- If `status` shows `succeeded` after a cancel, file an issue with the run id, the checkpoint timeline (`/runs/{run_id}/checkpoints`), and the trace (`/runs/{run_id}/trace`).

5.3 Run hangs at a workflow step

Workflow steps support `on_error: retry`, `max_retries`, and `timeout_seconds`. If a step has no timeout and the underlying provider/MCP call blocks, the step blocks too.

- Fix: set step-level timeouts, or set the global defaults
`KNEO_SERV_PROVIDER_TIMEOUT_SECONDS` / `KNEO_SERV_MCP_TIMEOUT_SECONDS`.

5.4 409 Conflict – idempotency_key_conflict

`Idempotency-Key` was reused with a different request body for the same scope.

[service/idempotency.py](#)

- Fix: pick a new key for the new request body, or reuse the same body for the original key. Idempotency records hash the canonical JSON of the request payload and replay the original response on match.

5.5 400 Bad Request – invalid_idempotency_key

`Idempotency-Key` headers must be 1–256 characters after trimming. [service/idempotency.py](#)

- Fix: shorten the key. UUIDs are sufficient.

6. Spec validation and compilation

6.1 SpecCompilationError with diagnostics

`SpecCompiler` raises this on either schema or semantic validation failure. [spec/compiler.py](#)

- Confirm: `kneo spec validate <path>` prints the same diagnostics with location info.
- Fix: address each diagnostic. Common causes:
 - Missing/extra fields in `version: v1` shape.
 - References to undefined components/tools.
 - Memory/guardrail policy mis-shape.
 - Migrate older specs with `kneo spec migrate <path> --output <new>`.

6.2 ValueError: Tool '<name>' has no implementation

The spec references a tool name that is not registered with the `ToolRegistry`. [spec/builder.py](#)

- Fix: register the tool (programmatically or via MCP import), or remove the reference. The default service registers example tools; toggle with `include_example_tools` when constructing `PlatformManager` directly.

6.3 Inline spec rejected with size error

Inline specs and overrides are bounded. [service/limits.py](#)

- Fix: tune the relevant limit (`KNEO_SERV_MAX_INLINE_SPEC_BYTES` , `KNEO_SERV_MAX_OVERRIDES_BYTES` , `KNEO_SERV_MAX_METADATA_BYTES` , `KNEO_SERV_MAX_BODY_BYTES`) or move the spec to a path on disk. Limits exist to keep the service from deserializing arbitrarily large payloads.

7. Observability

7.1 Structured logs missing request_id

The structured logging middleware always populates `request_id`; missing fields usually mean the log was emitted before `RequestLoggingMiddleware` attached, or you're reading raw unicorn access logs instead of the service logger.

- Fix: filter by logger name `kneo_serv` or by JSON log format. Clients can supply `X-Request-ID` to override the generated id; the service echoes it on the response.

7.2 OpenTelemetry not exporting

The SDK `OpenTelemetryMiddleware` only attaches when both `KNEO_SERV_OTEL_ENABLED=true` and the `[telemetry]` extra is installed.

- Fix: install `kneo-serv[deploy]` (Docker image) or `kneo-serv[telemetry]`, set `KNEO_SERV_OTEL_ENABLED=true`, and ensure standard OTEL exporter env vars (`OTEL_EXPORTER_OTLP_ENDPOINT`, etc.) are set.
- Tool arguments and results are **not** captured by default. Enable with `KNEO_SERV_OTEL_RECORD_ARGUMENTS=true` and/or `KNEO_SERV_OTEL_RECORD_RESULTS=true` only after you've confirmed the data classification allows payload capture.

7.3 Trace events missing for a run

Service-side trace events live in run metadata and at `/runs/{run_id}/trace`. They are emitted by `TracingMiddleware` and the in-process `Tracer`, independent of OTEL. Missing events most often mean the run was never executed (e.g. queued and abandoned) or the spec disabled tracing.

- Fix: confirm the run reached `running / succeeded`. If the workflow middleware list omits `TracingMiddleware`, restore it (the default chain includes it).

8. Human-in-the-loop

8.1 LockAcquisitionError on resume

A `POST /human-tasks/{continuation_id}/resume` failed because another caller currently holds the resume lock for the same continuation. [platform/manager.py](#)

- Confirm: the error body identifies the lock name. The first caller is still in flight.
- Fix: wait for the in-flight resume to complete; do not retry blindly. Use idempotency keys on resume to make retries safe.

8.2 Continuation expired or missing

If the continuation store was rotated (e.g. `.kneo/continuations` recreated, or PostgreSQL row deleted), `/human-tasks/{continuation_id}` returns 404.

- Fix: the run cannot be resumed. Start a new run.

9. Release and supply chain

For release-flow issues (mypy, pip-audit, build, tag, publish), follow [release_checklist.md](#) and [supply_chain_review.md](#). The release workflow at [.github/workflows/release.yml](#) emits the gate

that failed in its job summary.

What to capture before opening a bug

When a problem isn't covered above:

- Service version and commit (`pip show kneo-serv` , plus the git commit if installed from source).
- Environment context (`uname -a` , Python version, Postgres version if used).
- The `request_id` and `run_id` from logs.
- `GET /readyz` body.
- For run problems: `GET /runs/{run_id}` , `GET /runs/{run_id}/trace` , and `GET /runs/{run_id}/checkpoints` (redacted output is fine to share).
- For spec problems: the output of `kneo spec validate <path> --json` .

Audit events are accessible at `GET /audit-events` and frequently contain the operator action that preceded a fault.

Deployment smoke test

Source: docs/user/deployment_smoke.md

This smoke test validates a running service deployment through the public HTTP API. It uses the self-contained `examples/smoke_human_workflow.yaml` spec and covers health, readiness, auth behavior, spec validation, run creation, human resume, audit listing, credential inventory, and environment policy updates.

Compose stack

Prepare a production env file:

```
cp deploy/production.env.example deploy/production.env
```

`deploy/production.env` is intentionally ignored by source control. Replace all placeholder tokens and passwords before binding a deployment to a real network. For local CI/smoke runs, the documented placeholder values are used only to exercise the auth path.

Start the API and PostgreSQL:

```
docker compose --env-file deploy/production.env up --build -d
```

Run the smoke test against the unversioned routes:

```
python scripts/deployment_smoke.py \
  --base-url http://127.0.0.1:8000 \
  --api-key replace-admin-token \
  --operator-api-key replace-operator-token \
  --reviewer-api-key replace-reviewer-token \
  --viewer-api-key replace-viewer-token \
  --expect-auth
```

Run the same smoke test against the versioned routes:

```
python scripts/deployment_smoke.py \
  --base-url http://127.0.0.1:8000 \
  --api-prefix /v1 \
  --api-key replace-admin-token \
  --operator-api-key replace-operator-token \
  --reviewer-api-key replace-reviewer-token \
```

```
--viewer-api-key replace-viewer-token \
--expect-auth
```

Shut the stack down when finished:

```
docker compose --env-file deploy/production.env down
```

PostgreSQL coverage

The compose stack uses PostgreSQL by default through

`KNEO_SERV_DATABASE_URL=postgresql://...@db:5432/...`, so the smoke path above also validates PostgreSQL-backed run state, checkpoints, continuations, queue records, locks, audit events, and project metadata.

For a separately managed PostgreSQL database, start the API with `KNEO_SERV_DATABASE_URL` set and run the same `scripts/deployment_smoke.py` commands against that service URL.

Staging and remote smoke

Prepare a staging env file from the example:

```
cp deploy/staging.env.example deploy/staging.env
```

Replace every placeholder token, database password, provider key, and telemetry endpoint before use. For compose-based staging rehearsals, set `KNEO_SERV_ENV_FILE` so the API container reads the staging file:

```
python scripts/validate_staging_env.py deploy/staging.env
```

The validator fails if required staging settings are missing, scoped API roles are incomplete, payload telemetry capture is enabled, provider secret checks are disabled, or placeholder values remain.

For repeatable staging gates, render the local file from secret-backed environment variables instead of editing it by hand:

```
export KNEO_STAGING_API_KEYS="operator:<operator-token>;operator;reviewer:<reviewer-token>;reviewer;viewer:<viewer-token>;viewer"
export KNEO_STAGING_ADMIN_API_KEY=<admin-token>
export KNEO_STAGING_SPEC_SIGNING_KEY=<signing-key>
export KNEO_STAGING_DATABASE_URL=<postgresql-dsn>
export KNEO_STAGING_POSTGRES_PASSWORD=<compose-db-password>
export KNEO_STAGING_OTEL_EXPORTER_OTLP_ENDPOINT=<otel-endpoint>
```

```
export KNEO_STAGING_OPENAI_API_KEY=<openai-key>
export KNEO_STAGING_MCP_API_KEY=<mcp-key>
python scripts/render_staging_env.py --output deploy/staging.env
```

```
KNEO_SERV_ENV_FILE=./deploy/staging.env \
docker compose --env-file deploy/staging.env up --build -d
```

For a remote staging deployment, run the smoke script against the public staging URL with scoped keys:

```
export KNEO_STAGING_BASE_URL=https://staging.example.com
export KNEO_STAGING_OPERATOR_TOKEN=<operator-token>
export KNEO_STAGING_REVIEWER_TOKEN=<reviewer-token>
export KNEO_STAGING_VIEWER_TOKEN=<viewer-token>
python scripts/deployment_smoke.py --api-prefix /v1 --expect-auth
```

The operator key validates specs, creates runs, reads audit and credential inventory, and updates policy state. The reviewer key resumes the human task. The viewer key is optional; when supplied, the smoke verifies that policy writes are rejected with `403`.

The full staging release gate validates `deploy/staging.env`, derives the scoped smoke tokens from `KNEO_SERV_API_KEYS`, and runs the `/v1` remote smoke:

```
python scripts/staging_release_gate.py \
--env-file deploy/staging.env \
--base-url https://staging.example.com
```

The GitHub Actions `Staging Gate` workflow runs the same renderer and release gate from the `staging` environment secrets. Dispatch it with the deployed staging URL after the service is reachable.

See also

- [backup_and_recovery.md § Data-only restore into a clean volume](#) — the seeded recovery drill that pairs with the scoped Compose smoke above. Run the smoke first to populate run, checkpoint, audit, and policy state, then exercise the data-only restore to verify it survives a fresh volume.